

บทที่ 5

การพัฒนาระบบไฟร์สกรีน

5.1 แนวคิด

ระบบไฟร์สกรีนในโครงงานนี้เป็นการรวมความสามารถของส่วนไฟร์สกรีนจากโครงงาน ISAG Firewall Suite 2548 และความสามารถในส่วนเกตเวย์ฮันนีพอต จากโครงงาน ISAG Honeypot Suite 2548 เข้าไว้ด้วยกัน โดยจะพัฒนาและออกแบบให้เป็นเกตเวย์ไฟร์วอลล์ทำหน้าที่ป้องกันการโจมตีได้ตามกฎที่รับมาจากเครื่องไฟร์สแตชั่นซึ่งเก็บไว้ในแอ็คทีฟไดเรกทอรีดาต้าเบส โดยสามารถเข้าถึงผ่านทางโปรโตคอล LDAP โดยอาศัยไลบรารี OpenLDAP ซึ่งจะทำให้การตรวจสอบแพ็คเก็ตที่ผ่านเข้ามาในระบบเครือข่ายว่าเป็นไปตามกฎที่ตั้งไว้หรือไม่ แต่ถ้าไม่ตรงกับกฎใด ๆ เลย แพ็คเก็ตจะถูกตรวจสอบโดยอาศัยระบบตรวจจับผู้บุกรุกอีกทีหนึ่ง เพื่อทำการจำแนกผู้บุกรุกออกจากผู้ใช้ปกติ ถ้าแพ็คเก็ตนั้นเข้าข่ายเป็นผู้บุกรุก ระบบจะทำการเปลี่ยนให้ผู้บุกรุกไปใช้งานในส่วนของไฟร์แทรป (FireTrap) แทน และส่งข้อมูลการบุกรุกเกี่ยวกับการบุกรุกและการกระทำของผู้บุกรุกนั้น ๆ ไปยังส่วนระบบล็อกมอนิเตอร์ต่อไป

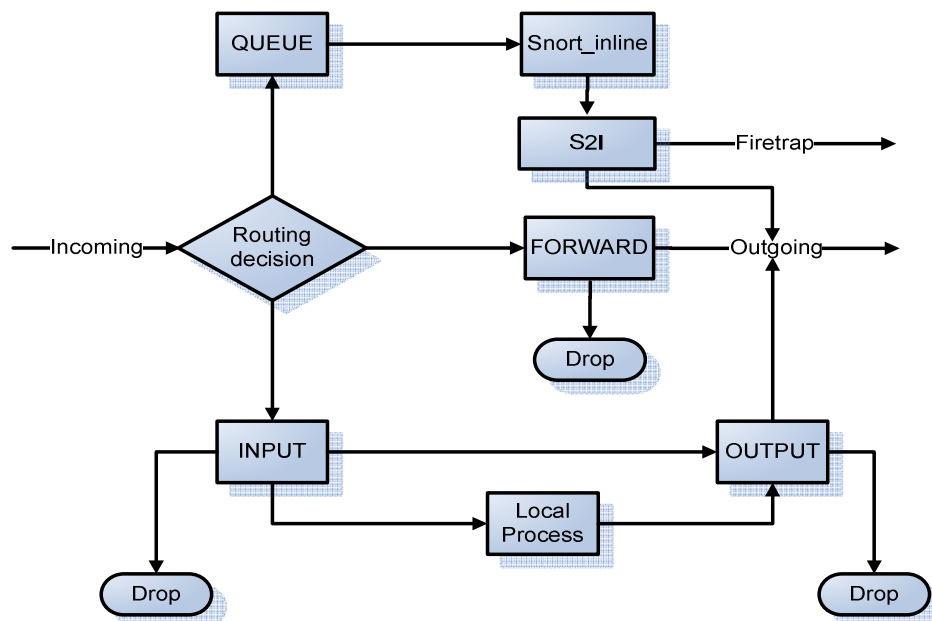
5.2 ขอบเขตความสามารถ

ไฟร์สกรีนจะทำหน้าที่เป็นเกตเวย์ไฟร์วอลล์ของระบบทั้งหมด ซึ่งในการเชื่อมต่อทุก ๆ ส่วนจะต้องผ่านทางเกตเวย์นี้ ทำให้ไฟร์สกรีนสามารถควบคุมการเชื่อมต่อเกือบทั้งหมดของระบบไว้ได้ โดยในการดูแลการเข้าออกของแพ็คเก็ต จะทำการจำแนกและตรวจสอบแพ็คเก็ตหรือการเชื่อมต่อต่างๆ ที่เข้ามาในระบบตามกฎของไฟร์วอลล์ที่กำหนดโดยผู้ดูแลระบบซึ่งรับมาจากไฟร์สแตชั่น และกระทำ Action ต่าง ๆ กับแพ็คเก็ตนั้นตามที่กำหนดไว้ แต่สำหรับแพ็คเก็ตที่ไม่ตรงกับกฎใด ๆ ของไฟร์วอลล์จะถูกส่งมาตรวจสอบว่ามีพฤติกรรมเข้าข่ายเป็นผู้บุกรุกหรือไม่ โดยใช้ Signature ของโปรแกรมตรวจจับผู้บุกรุก เช่น สนอร์ตอินไลน์ (Snort_inline) ซึ่งเป็นเครื่องมือประเภท IDS (Intrusion Detection System) โดยหากว่าการเชื่อมต่อใดมีลักษณะตรงตาม Signature ของโปรแกรมตรวจจับผู้บุกรุกจะถือว่าเข้าของการเชื่อมต่อนั้นมีพฤติกรรมเข้าข่ายเป็นผู้บุกรุก และจะต้องทำการเปลี่ยนทิศทางการเชื่อมต่อดังกล่าวไปยังระบบจำลองที่ชื่อ ไฟร์แทรป (FireTrap) เพื่อทำการบันทึกพฤติกรรมของผู้บุกรุกที่กระทำต่อระบบต่อไป

แต่ในการทำงานเพียงลำพังของระบบตรวจจับผู้บุกรุกนั้นไม่เพียงพอที่จะจัดการให้ผู้บุกรุกเข้าสู่ระบบจำลองไฟร์แทรปที่จัดเตรียมไว้ได้ เนื่องจากระบบตรวจจับผู้บุกรุกนั้นไม่มีความสามารถเพียงพอในการสร้างเส้นทางในการเชื่อมต่อ ซึ่งความสามารถนี้เป็นความสามารถของไฟร์วอลล์ ที่มีชื่อเรียกว่า ไอพีเทเบิล (Iptables) ซึ่งจะเป็นตัวจัดการการเชื่อมต่อสื่อสารเพื่อให้เป็นไปตามความ

ต้องการของผู้พัฒนา แต่ในการทำงานจริงนั้น ทั้งสองโปรแกรมนี้ไม่สามารถพูดคุยเพื่อควบคุมกันได้ เนื่องจากว่าไอพีเทเบิล เป็นโปรแกรมที่ทำงานในส่วนของเคอร์เนลโหมด (kernel mode) แต่ Snort 2.0 นั้นเป็นโปรแกรมที่ทำงานในส่วนยูสเซอร์โหมด (User mode) ดังนั้นจึงได้มีการพัฒนาตัวกลางที่ทำหน้าที่สื่อสารระหว่างระบบตรวจจับผู้บุกรุกกับไฟร์วอลล์ไอพีเทเบิลขึ้นมาเพื่อเป็นตัวกลางที่จะอ่านค่าจากการแจ้งเตือนจากระบบตรวจจับผู้บุกรุก และนำค่าได้มานั้นแปลงเป็นคำสั่งของไอพีเทเบิลอย่างเหมาะสมเพื่อสั่งให้จัดส่งขึ้นข้อมูลไปยังปลายทางที่ต้องการ ซึ่งทางผู้พัฒนาในการพัฒนาโปรแกรม S2I (Snort command to Iptables) ขึ้นมาเพื่อทำหน้าที่ดังกล่าว

นอกจากนี้โปรแกรม S2I จะคอยตรวจสอบความอ่อนแอของเครื่องกับดักในส่วนของไฟร์แตรปอยู่ตลอดเวลาว่ามีความอ่อนแอหรือถูกเปลี่ยนแปลงโดยผู้บุกรุกหรือไม่ โดยถ้าถึงจุดที่เครื่องกับดักมีความเสียหายก็จะทำการหยุดให้บริการเครื่องนั้นโดยอัตโนมัติและจะทำการส่งเครื่องกับดักเครื่องใหม่ขึ้นมาทำงานต่อ เพื่อป้องกันการใช้เครื่องกับดักเป็นฐานในการโจมตีระบบอื่น



รูปที่ 5.1 แผนผังการทำงานของระบบไฟร์สกรีน

5.3 เทคโนโลยีที่ใช้ในการพัฒนาโปรแกรม

5.3.1 ไอพีเทเบิล (Iptables)

ไอพีเทเบิลนั้นเป็นโปรแกรมไฟร์วอลล์แบบสเตทฟูลอินสเปกชันที่ทำงานในเคอร์เนลโหมด โดยไอพีเทเบิลจะมีหน้าที่ในการจัดการเชื่อมต่อ และการอนุญาตหรือไม่อนุญาตให้ขึ้นข้อมูลต่างๆ สามารถผ่านเข้าออกระบบเครือข่ายได้ ในโครงการนี้ได้นำไอพีเทเบิลมาใช้ในการตรวจสอบขึ้นข้อมูลหรือการเชื่อมต่อต่าง ๆ ที่เข้ามาตามกฎที่ตั้งไว้จากไฟร์สแตชันโดยผู้ดูแลระบบ และทำการสร้างเส้นทางให้เหมาะสม นอกจากนี้ยังนำโมดูล ip_queue ของไอพีเทเบิลมาทำงานร่วมกับ

โปรแกรมสนอร์ตอินไลน์เพื่อตรวจสอบชิ้นข้อมูลที่ไม่ตรงตามกฎใด ๆ ของไฟร์วอลล์และทำการจำแนกชิ้นข้อมูลนั้น ๆ ว่ามีพฤติกรรมเข้าข่ายเป็นผู้บุกรุกหรือไม่

รูปแบบของคำสั่งในการใช้งานไอพีเทเบิล

```
iptables [table] <command> <match> <target/jump>
```

ไอพีเทเบิลมีรูปแบบการใช้งานดังข้างต้น โดยกฎที่เขียนขึ้นจะเป็นตัวบอกเคอร์เนลว่าให้กระทำอย่างไร (action) ในกรณีที่พบแพ็คเก็ตที่ตรงตามกฎที่ระบุไว้

■ Command

- A เพิ่มกฎใหม่ต่อท้าย chain (Append rule)
- D ลบกฎ (Delete rule)
- I เพิ่มกฎใหม่ใน chain (Insert rule)
- R แทนที่กฎเดิมด้วยกฎใหม่ (Replace rule)
- L แสดงกฎใน chain (ถ้าไม่ระบุ chain จะแสดงกฎใน filter table ทั้งสาม built-in chain)
- F ลบกฎทั้งหมดใน chain ทิ้ง
- Z ใช้ reset byte counter สำหรับทุกกฎใน chain ที่กำหนด
- N ใช้สร้าง chain ใหม่
- X ลบ chain ใช้กับ user-defined chain ที่ไม่มีกฎ แต่ไม่สามารถลบ built-in chain ได้
- P เปลี่ยน default policy ของ chain ค่าที่ใช้กับ command นี้คือ ACCEPT, DROP
- E ใช้เปลี่ยนชื่อ chain ใหม่

ซึ่งการใช้ command ข้างต้นนั้นสามารถใช้ร่วมกับออปชั่นบางอย่างได้ คือ

-v (--verbose) ใช้ร่วมกับ -L, -A, -I, -D, -R เพื่อให้แสดงจำนวน byte ที่ match กับ กฎออกมา (หน่วยที่แสดงออกมานั้นอาจจะเป็น K, M, G)

-x (--exact) ใช้ร่วมกับ -L และ -v เพื่อให้แสดงจำนวนแพ็คเก็ตและจำนวนของ byte ข้อมูลที่ match โดยไม่แสดงผลในหน่วยของ K, M, G

-n (--numeric) ใช้ร่วมกับ -L เพื่อสั่งให้ไอพีเทเบิล แสดงข้อมูลหมายเลขไอพีแอดเดรส และ หมายเลขพอร์ตเป็นตัวเลขเท่านั้น

--line-numbers ใช้ร่วมกับ -L เพื่อแสดงเลขบรรทัดของกฎซึ่งตัวเลขที่แสดงนี้จะสามารถใช้ได้กับคำสั่งแทรกกฎ ที่ระบุเป็นลำดับที่ของกฎ

- Match

- การระบุหมายเลขไอพีแอดเรสต้นทางและหมายเลขไอพีแอดเรสปลายทาง

สามารถระบุไอพีแอดเรสต้นทางของแพ็คเก็ตโดยใช้ -s หรือ --source หรือ --src และ ไอพีแอดเรสปลายทางใช้ -d หรือ --destination หรือ --dst ในการระบุไอพีแอดเรสนั้นสามารถทำได้ 3 แบบด้วยกันคือ

1. ใช้ชื่อเต็มแทน เช่น localhost หรือ www.ce.kmitl.ac.th
2. ระบุไอพีแอดเรสโดยตรง เช่น 127.0.0.1 หรือ 161.246.4.7
3. ระบุเป็นกลุ่มของไอพีแอดเรส เช่น 161.246.5.0/24 ซึ่งหมายถึงไอพีแอดเรสดังแต่ 161.246.5.0 – 161.246.5.255 หรือ 161.246.5.0 /255.255.255.0 แทน 161.246.5.0 /24 ได้

- การทำ Inversion

ทำได้โดยใช้เครื่องหมาย ! นำหน้า argument ที่ต้องการ (! หมายถึง NOT) เช่น -p ! TCP

- การระบุโปรโตคอล

สามารถระบุโปรโตคอลที่ต้องการได้ดังนี้คือ TCP, UDP, ICMP และสามารถใช้ได้ทั้งตัวอักษรเล็กหรือใหญ่ (ใช้ได้ทั้ง tcp และ TCP) เช่น -p TCP หรือ -p ! tcp

- การระบุ interface

-i หรือ --in-interface ตามด้วยชื่ออินเทอร์เฟซ ใช้เพื่อระบุอินเทอร์เฟซขาเข้าซึ่งหมายถึงว่าแพ็คเก็ตที่จะตรงกับกฎนี้ต้องเข้ามาจากอินเทอร์เฟซที่กำหนด เช่น -i eth0 หมายความว่า ทุกแพ็คเก็ตที่เข้ามาทาง eth0 จะตรงกับกฎนี้

-o หรือ --out-interface ตามด้วยชื่อของอินเทอร์เฟซ ใช้เพื่อระบุอินเทอร์เฟซขาออก ซึ่งหมายถึงว่าแพ็คเก็ตที่จะตรงกับกฎนี้ กำลังจะเดินทางผ่านอินเทอร์เฟซที่ระบุไว้ เช่น -o eth1 หรือ -o ! eth1 จะตรงกับกฎนี้

- fragment packet

ในการส่งข้อมูลผ่านเครือข่ายนั้นเป็นเรื่องปกติที่จะเกิดการ แฟร็กเมนต์ (fragment) ของแพ็คเก็ตเนื่องจากขนาดของแพ็คเก็ตมีขนาดใหญ่เกินไปที่จะส่งไปในครั้งเดียว จำเป็นต้องมีการแบ่งแพ็คเก็ตออกเป็นหลายๆชิ้นทยอยส่งไป ซึ่งเรียกกันว่าการทำ fragmentation โดยเครื่องปลายทางจะทำหน้าที่ประกอบแพ็คเก็ตที่ผ่านการทำแฟร็กเมนต์ ให้มารวมกันเป็นแพ็คเก็ตที่สมบูรณ์ดังเดิม ข้อมูลของแฟร็กเมนต์แพ็คเก็ตจะมีเฮดเดอร์ที่สมบูรณ์แค่แพ็คเก็ตแรกเท่านั้น แพ็คเก็ตที่ตามมาจะมีเฮดเดอร์แค่บางส่วนคือ ไอพีแอดเรส ไม่มีข้อมูลของโปรโตคอลแบบมาด้วย ดังนั้นการตรวจสอบข้อมูลเฮดเดอร์ของ TCP, UDP, ICMP จึงไม่สามารถทำได้ในแพ็คเก็ตที่สองเป็นต้นมา

โดยปกติแล้วมักจะปล่อยให้แพ็คเก็ตที่ทำการแฟร็กเมนต์มาแล้วผ่านไป เนื่องจากถ้าสามารถ DROP แพ็คเก็ตตัวแรกได้แล้ว มันก็ไม่สามารถถูกประกอบที่เครื่องปลายทางได้ แต่ทั้งนี้แฟร็กเมนต์แพ็คเก็ต ที่ถูกปล่อยไปดังกล่าวอาจจะทำให้เครื่องที่ได้รับไม่สามารถทำงานต่อได้

- TCP extension

ถ้ามีการเรียกใช้ `-p tcp` ตัว TCP extension จะถูกโหลดมาใช้งานโดยอัตโนมัติ โดยมีตัวเลือกให้เลือกใช้งานดังนี้

`--tcp-flags mask flags` : mask นั้นหมายถึง flag ที่ต้องการตรวจสอบ และ flag เป็นตัวที่บ่งชี้ว่า flag ใดต้องถูก set บ้าง

`--source-port` หรือ `--sport` สามารถใช้ได้ทั้งตัวเลขและตัวอักษร ระบุเป็นพอร์ตเดี่ยว หรือ ช่วงของพอร์ตได้

`--destination-port` หรือ `--dport` มีรูปแบบการใช้งานเช่นเดียวกันกับ `--sport`

- UDP extension

เช่นเดียวกับ TCP ตัว UDP extension มีตัวเลือกให้เลือกใช้เพียงแค่ 2 อย่างเท่านั้นคือ `--source-port` (`--sport`) และ `--destination-port` (`--dport`) โดยต้องระบุ `-p udp` ด้วย

- ICMP extension

โดยการระบุ `-p icmp` ก็สามารถใช้งาน ICMP extension ได้ โดยมีตัวเลือกให้เลือกคือ `--icmp-type` เช่น `--icmp-type host-unreachable` (หรือใช้เลข 3 แทนได้) นอกจากนี้ยังสามารถระบุ type/code ได้ เช่น 3/3 ซึ่งหมายถึง port unreachable

- Match Extension

เป็น netfilter package อื่นๆ รูปแบบการใช้งานให้ใช้ `-m` แล้วตามด้วย match ที่ต้องการ มีตัวเลือกให้เลือกใช้งานดังต่อไปนี้

mac รูปแบบการใช้งาน: `-m mac` ใช้ตรวจสอบ source MAC address ว่าตรงกับค่าที่ระบุไว้หรือไม่ มีประโยชน์สำหรับ *PREROUTING* และ *INPUT chain*

limit รูปแบบการใช้งาน: `-m limit` ใช้เพื่อจำกัดจำนวนของการ match ที่อาจจะมากเกินไป เป็นประโยชน์สำหรับกฎที่วางไว้ตอนท้ายสุดของ chain (ใช้รวมกันกับ DROP policy) ซึ่งส่วนใหญ่เป็นกฎที่ใช้เก็บข้อมูลลงล็อกไฟล์ ซึ่งถ้าผู้บุกรุกส่งแพ็คเก็ตที่ไม่เข้าข่ายกฎใดๆ ใน chain จนกระทั่งมาถึงกฎที่กำหนดที่เก็บล็อกนี้ ถ้าแพ็คเก็ตที่เข้ามามีจำนวนมาก อาจจะทำให้ฮาร์ดดิสก์เต็มได้ ดังนั้นจึงต้องใช้จำกัดจำนวนในการเก็บข้อมูลลงล็อก ซึ่งมีตัวเลือกให้ใช้งานดังนี้คือ

State Match รูปแบบการใช้งานคือ `-m state` หรือ `--match state` เป็นโมดูลที่ใช้ประโยชน์ได้เป็นอย่างดี มีตัวเลือกให้ใช้งานดังนี้

NEW – แพ็คเก็ตที่เป็นตัวสร้างการเชื่อมต่อใหม่

ESTABLISHED – แพ็คเก็ตที่เกี่ยวข้องกับการเชื่อมต่อที่สร้างไว้แล้ว

RELATED – แพ็คเก็ตที่เกี่ยวข้องกับการเชื่อมต่อที่สร้างไว้แล้ว แต่ไม่ใช่ส่วนหนึ่งส่วนใดของการเชื่อมต่อ

INVALID – เป็นแพ็คเก็ตที่ไม่เกี่ยวข้องกับส่วนอื่นเลย

- Target

เป็นส่วนที่ระบุ action ที่กระทำกับแพ็กเก็ตต่างๆ ที่ตรงกับกฎที่ได้ตั้งขึ้นซึ่งมี target ในแบบต่างๆดังนี้

- ACCEPT อนุญาตให้แพ็กเก็ตที่ตรงกับกฎนี้สามารถผ่านไป
- DROP แพ็กเก็ตที่ตรงกับกฎนี้ให้ทำการ Drop ทิ้งหรือไม่ให้ผ่าน
- LOG เป็นโมดูลที่มีความสามารถในการเก็บข้อมูลลงล็อก
- REJECT คล้ายกับ DROP แต่จะมีการส่ง ICMP port unreachable กลับไปยังผู้ส่ง
- RETURN ออกจากเชน (chain) ปัจจุบันกลับไปยังเชนที่ทำการเรียกมา
- QUEUE เป็นเชนพิเศษใช้สำหรับส่งต่อแพ็กเก็ตไปยังแอปพลิเคชันที่เขียนขึ้นมารองรับโดยเฉพาะ โดยจะต้องมี queue handler และแอปพลิเคชันเป็นส่วนประกอบที่จะทำงานร่วมกัน
- User-defined chain ผู้ใช้สามารถสร้างเชนใหม่ได้โดยต้องใช้ตัวอักษรตัวเล็กทั้งหมดสำหรับเชนที่สร้างขึ้นใหม่ เมื่อแพ็กเก็ตตรงกับกฎที่เป็น user-defined chain แพ็กเก็ตจะถูกนำไปตรวจสอบใหม่โดย user-defined chain นั้นๆและถ้าในเชนไม่มีการตัดสินใจ action ใดๆ ตัวแพ็กเก็ตก็สามารถย้อนกลับมายังกฎถัดไปในเชนเริ่มต้นได้

5.3.2 สนอร์ตอินไลน์ (Snort_inline)

สนอร์ตอินไลน์เป็นโปรแกรมไอดีเอสที่ทำงานในยูสเซอร์โหมด ซึ่งจะทำหน้าที่เป็นระบบตรวจจับผู้บุกรุก ในโครงการนี้ได้นำสนอร์ตอินไลน์ มาใช้ในการจำแนกว่าผู้ใดเป็นผู้ใช้งานปกติและผู้ใดเป็นผู้บุกรุกระบบ ซึ่งจะจัดจำแนกตามข้อมูลหรือกฎที่ผู้ดูแลระบบใช้ ซึ่งในที่นี้โปรแกรมจะทำการอ่านค่าขึ้นข้อมูลที่ส่งมาทางมอดูล ip_queue ของไอพีเทเบิลแล้วนำมาเปรียบเทียบกับกฎที่ได้ตั้งไว้ ว่าเข้าข่ายเป็นการโจมตีหรือไม่ และทำการแจ้งเตือนในกรณีที่การตรวจสอบมีผลออกมาว่าขึ้นข้อมูลที่ส่งมามีลักษณะของการโจมตีระบบ

สนอร์ตอินไลน์เป็นโปรแกรมที่พัฒนามาจากโปรแกรมสนอร์ต (Snort) ซึ่งโปรแกรมสนอร์ตอินไลน์จะรับแพ็กเก็ตจากโปรแกรมไอพีเทเบิล ผ่านทาง libipq แทนที่ของเดิมที่จะรับจาก libpcap และกฎที่ใช้ในสนอร์ตอินไลน์จะเป็นรูปแบบของกฎใหม่ โดยมีการทำงาน drop, sdrop, reject โดยสั่งงานไอพีเทเบิลได้ 3 คำสั่งดังกล่าว ทำให้สนอร์ตอินไลน์จึงเหมือนทำหน้าที่เป็น IPS (Intrusion Prevention System)

การกระทำอันเนื่องมาจากกฎที่ใช้

Drop - The drop rule type

จะไปสั่งให้ออฟเพิลทำการ drop แพ็คเก็ต และทำการเก็บล็อกไว้

Reject - The reject rule type

จะไปสั่งให้ออฟเพิลทำการ drop แพ็คเก็ต และทำการเก็บล็อกไว้ พร้อมทั้งส่ง TCP reset หรือ UDP reset กลับไป แล้วแต่กรณี

Sdrop - The sdrop rule type

จะไปสั่งให้ออฟเพิลทำการ drop แพ็คเก็ต โดยไม่ทำการบันทึกใดๆ

ลำดับในการพิจารณากฎของSnort inline

->activation->dynamic->drop->sdrop->reject->alert->pass->log

สามารถใช้ทางเลือก -o เพื่อเปลี่ยนลำดับเป็น

->activation->dynamic->pass->drop->sdrop->reject->alert->log

Stream4 option การทำงานใหม่ใน Snort inline

ซึ่งสามารถเปิดการใช้งานในทางเลือก ได้อีก 2 ความสามารถดังนี้

inline_state (no arguments)

เมื่อเปิดใช้งาน สนอร์ตอินไลน์จะทำการdrop แพ็คเก็ตที่ไม่เกี่ยวข้องหรือไม่

สอดคล้องกับ TCP session ที่มีอยู่ รวมถึงแพ็คเก็ตที่มี TCP initiator ผิดรูปแบบ

midstream_drop_alerts (no arguments)

โดยคำกำหนดเริ่มต้นสำหรับสนอร์ตอินไลน์จะทำการdrop โดยไม่แสดงการเตือน

ใดๆ ถ้าหากเราต้องการให้มีการแจ้งเตือนจะต้องเปิดโหมดการทำงานอันนี้ด้วย

ความสามารถในการแก้ไขข้อมูลในแพ็คเก็ต

นั่นคือเราสามารถสั่งให้มีการตรวจสอบ ข้อมูลที่ผ่านเข้าออกที่ผ่านตัวสนอร์ตอินไลน์ว่ามีค่าที่ตรงกับที่เราตั้งไว้หรือไม่ และถ้ามีจะทำการเปลี่ยนให้เป็นไปตามรูปแบบที่เราต้องการ เช่น

```
alert tcp any any <> any 80 (msg: "tcp replace"; content:"GET"; replace:"BET";)
```

```
alert udp any any <> any 53 (msg: "udp replace"; content: "yahoo"; replace: "xxxxx");
```

ดังตัวอย่าง สนอร์ตอินไลน์จะดูข้อมูลที่เป็นTCP ที่ใช้งาน port 80 โดยถ้าพบว่ามีส่วนที่ตรงกับคำว่า GET โปรแกรมจะให้เป็น BET และถ้าพบข้อมูลที่เป็น UDP ที่ใช้งาน port 53 ที่มีข้อมูลตรงกับคำว่า yahoo ก็ให้เป็น xxxx

5.3.3 S2I (snort command to iptables)

เป็นโปรแกรมที่สร้างมาจากภาษาเพิร์ลที่พัฒนาขึ้น เพื่อให้สามารถแปลงการแจ้งเตือนจากโปรแกรมสนอร์ตอินไลน์ (Snort_inline) ไปเป็นคำสั่งของโปรแกรมไฟร์วอลล์ไอพีเทเบิล (Iptables) เพื่อโปรแกรมไอพีเทเบิลจะได้จัดเส้นทางให้กับชั้นข้อมูลที่มาจากผู้ใช้งานปรกติผ่านไปยังส่วนที่เป็นเซิร์ฟเวอร์จริง และในส่วนของผู้นุกรูเองนั้นก็ให้ส่งไปยังเครื่องกับดักที่ได้เตรียมไว้ ซึ่งสาเหตุที่ต้องมีเอสทูไอนั้นเนื่องมาจากโปรแกรมไอดีเอสและโปรแกรมไอพีเทเบิลที่ใช้ในโครงการนี้จะทำงานในโหมดที่ต่างกันจึงติดต่อกันเองโดยตรงไม่ได้ ดังนั้นจึงต้องมีโปรแกรมสื่อกลางช่วยในการติดต่อกัน

นอกจากเป็นโปรแกรมในการสร้างเส้นทางใหม่ให้ผู้บุกรุกแล้ว ยังทำหน้าที่เป็นเซอร์วิสที่คอยตรวจสอบสถานะของเครื่องกับดักในระบบไฟร์แทรป (FireTrap) และจะทำการหยุดการทำงานของเครื่องกับดักหากเครื่องกับดักมีความอ่อนแอเกินกว่าที่จะสามารถควบคุมพฤติกรรมของผู้บุกรุกได้ โดยในการทำงานโปรแกรมจะไปอ่านข้อกำหนดสำหรับเครื่องกับดักที่ผู้ดูแลระบบกำหนดไว้ในฐานข้อมูลแล้วทำการตรวจสอบเครื่องกับดักว่ามีสภาพตรงตามเงื่อนไขที่กำหนดหรือยัง เช่นมีการเปลี่ยนแปลง Password ในเครื่องไฟร์แทรปหรือไม่ หากเครื่องกับดักมีสภาพถึงตามที่กำหนดไว้ โปรแกรมจะทำการหยุดการทำงานของเครื่องกับดักในระบบไฟร์แทรปโดยอัตโนมัติ และทำการส่งเครื่องกับดักตัวใหม่ขึ้นมาทำงาน เพื่อป้องกันไม่ให้ผู้บุกรุกใช้เครื่องกับดักเป็นฐานที่มั่นในการโจมตีระบบอื่น ๆ ซึ่งจะก่อให้เกิดความเสียหายต่อระบบจริงขององค์กรต่อไป

5.3.4 Oinkmaster

เป็นโปรแกรมที่ช่วยในการอัปเดต Signature ของโปรแกรมสนอร์ตอินไลน์โดยอัตโนมัติจากเว็บไซต์ต่างๆ ที่มี Signature ของสนอร์ตอินไลน์อยู่ ซึ่งผู้ดูแลระบบสามารถทำการกำหนดเว็บไซต์ที่ต้องการดาวน์โหลดได้แต่จะต้องแน่ใจว่าเว็บไซต์ต่าง ๆ นั้นน่าเชื่อถือ ซึ่งทางผู้พัฒนาได้ใช้โปรแกรมนี้เพื่อช่วยในการอัปเดตและแจกจ่ายกฎของสนอร์ตอินไลน์ไปยังส่วนต่างๆ ของระบบที่ต้องมีการจำแนกแพ็คเก็ตผู้บุกรุก

ในการใช้งาน Oinkmaster ทำได้โดยติดตั้งแพ็คเกจ Oinkmaster เพื่อช่วยในการดาวน์โหลดกฎจากเว็บไซต์ต่าง ๆ แบบอัตโนมัติ โดยขั้นตอนการติดตั้งแพ็คเกจ Oinkmaster สามารถทำได้ดังนี้

- แดกไฟล์แพ็คเกจ Oinkmaster ที่ดาวน์โหลดมาแล้วทำการย้ายไฟล์ oinkmaster.pl และ oinkmaster.conf ไปไว้ที่ใดเรกทอรีก็ได้
- แก้ไขไฟล์ oinkmaster.conf ที่บรรทัด 61 โดยแทนที่ <oinkcode> ด้วย key ที่ได้จากการลงทะเบียนทางเว็บไซต์ โดยแก้ไขไฟล์ดังนี้

url = http://www.snort.org/pub-bin/oinkmaster.cgi/<oinkcode>/snortrules-snapshot-2.3.tar.gz
เป็น

url = http://www.snort.org/pub-bin/oinkmaster.cgi/587cba039g9f03300f9e98b/snortrules-snapshot-2.3.tar.gz

หรือหากไม่มี key ที่ลงทะเบียนจากเว็บไซต์ www.snort.org ก็สามารแก้ไขโดยเพิ่ม URL ที่เป็น URL สำหรับดาวน์โหลด Signature แบบ Community-rules ได้

- ทดสอบการดาวน์โหลดกฎได้โดยสั่ง /etc/oinkmaster.pl -o /ect/snort_inline/rules/

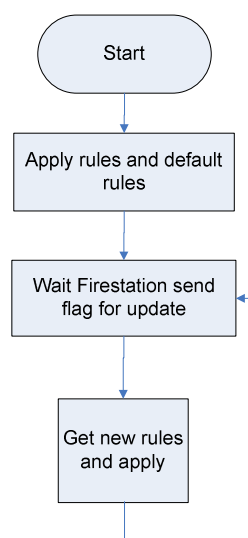
5.4 การทำงานของโปรแกรม

5.4.1 การทำงานร่วมกับไฟร์สแตชั่น

ไฟร์สกรีนมีระบบปฏิบัติการเป็นลินุกซ์ ซึ่งในที่นี้ใช้ดิสโทรเดเบียน (debian) โดยทำงานร่วมกับแอคทีฟไดเรกทอรีของไมโครซอฟท์บนไฟร์สแตชั่นนั้นจะใช้แพ็คเกจของ OpenLDAP โดยตัวโปรแกรมไฟร์สกรีนจะต้องใช้ไลบรารีของ OpenLDAP ในการเขียนโปรแกรมทำงานร่วมกับแอคทีฟไดเรกทอรี

5.4.1.1 การดึงกฎจากไฟร์สแตชั่น

สำหรับกฎของไฟร์วอลล์ที่เก็บไว้ในแอคทีฟไดเรกทอรีบนไฟร์สแตชั่นนั้นถูกเก็บไว้ได้ dn คือ "cn=firescreen, cn=computers, dc=hephaestus, dc=com" ภายในแอตทริบิวต์ชื่อ firewallRule ดังนั้นจึงสามารถเข้าถึงได้โดยใช้ฟังก์ชัน ldap_search_s() กำหนดขอบเขตของการค้นหาเป็น "cn=firescreen, cn=computers, dc=hephaestus, dc=com" จากนั้นจึงดึงค่าออกจากแอตทริบิวต์ firewallRule จนครบทุกค่าและเก็บค่าไว้รอใช้งานในอาร์เรย์



รูปที่ 5.2 การรับกฎจากไฟร์สแตชั่น

จากรูปแบบการทำงานของไฟร์สกรีนที่กล่าวถึงในหัวข้อก่อนหน้านี้ เมื่อกฎมีการถูกแก้ไข ไฟร์สแตชันจะส่งสัญญาณมายังไฟร์สกรีนเพื่อติดต่อไปยังแอ็กทิฟไดเรกทอรีเพื่อทำการดึงกฎมาจากแอตทริบิวต์ Firewallrule

5.4.1.2 การนำมาใช้งานกับไอพีเทเบิล

กฎที่ดึงมาจากทางไฟร์สแตชันนั้นจะอยู่ในรูปแบบดังต่อไปนี้

SRC:MASK:DST:MASK:PROTOCOL:SPORT:DPORT:ICMP-TYPE:ZONE:TARGET

SRC	ไอพีแอดเดรสต้นทาง
MASK	เน็ตมาสก์ของไอพีแอดเดรสต้นทาง
DST	ไอพีแอดเดรสปลายทาง
MASK	เน็ตมาสก์ของไอพีแอดเดรสปลายทาง
PROTOCOL	โพรโตคอล (TCP, UDP, ICMP หรือ any)
SPORT	พอร์ตต้นทาง (เมื่อระบุโพรโตคอลเป็น TCP หรือ UDP เท่านั้น)
DPORT	พอร์ตปลายทาง (เมื่อระบุโพรโตคอลเป็น TCP หรือ UDP เท่านั้น)
ICMP-TYPE	หมายเลข ICMP-TYPE (เมื่อระบุโพรโตคอลเป็น ICMP เท่านั้น)
ZONE	โซนที่กฎนี้จะมีผลในการใช้งาน
TARGET	ปลายทางของแพ็กเก็ตที่แมชท์กับกฎนี้

ตารางที่ 5.1 แสดงความหมายของแต่ละค่า

โดยก่อนที่จะสามารถนำมาใช้งานกับไฟร์สกรีนได้จะต้องแปลงกฎที่รับมาจากไฟร์สแตชันที่มีรูปแบบดังกล่าวให้อยู่ในรูปแบบของไอพีเทเบิลเสียก่อน โดยผ่านฟังก์ชันแปลง (parser) ซึ่งจะแยกแต่ละส่วนออกจากกันโดยมีเครื่องหมายโคลอน (:) เป็นตัวแบ่ง โดยนำเอาแต่ละส่วนมาต่อเข้าด้วยกันตามรูปแบบคำสั่งของไอพีเทเบิล ส่วนกฎจะถูกเพิ่มเข้าเชน (CHAIN) ในของไอพีเทเบิลจะพิจารณาจาก ZONE และ TARGET ที่ระบุเป็น DROP ทั้งหมดจะถูกระบุ TARGET ในคำสั่งไอพีเทเบิลเป็น -j LOG-CHAIN ซึ่งจะส่งแพ็กเก็ตที่ตรงกับกฎเข้าไปยังเชนชื่อ LOG-CHAIN เพื่อที่หากแพ็กเก็ตที่ถูกละทิ้ง (DROP) จะมีการเก็บข้อมูลลงล็อกไฟล์

```
Chain LOG-CHAIN (4 references)
num target prot opt source destination limit: avg 1/sec
1 LOG all -- anywhere anywhere
burst 5 LOG level info prefix 'FIREScreen '
2 DROP all -- anywhere anywhere
```

รูปที่ 5.3 กฎในเชน LOG-CHAIN

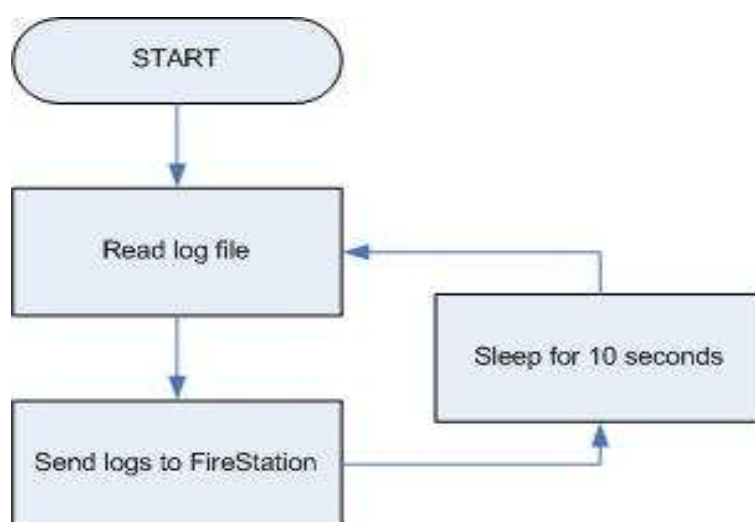
กฎแรกจะทำการเก็บข้อมูลทุกแพ็คเกจที่ผ่านเข้ามาในเซิร์ฟเวอร์นี้ลงล็อกไฟล์ในอัตรา 1 แพ็คเกจต่อวินาที (limit: avg 1/sec burst 5) โดยทุกค่าที่ถูกเก็บลงล็อกไฟล์จะมีคำว่า "FIREScreen " (prefix 'FIREScreen') ขึ้นต้น เพื่อประโยชน์ในการคัดแยกล็อกในภายหลัง เนื่องจากกระบวนการเก็บเหตุการณ์ล็อกไฟล์นั้นถูกจัดการโดย syslogd ซึ่งไม่ได้ถูกจัดการโดยไอฟีเทเบิลเองทำให้ในล็อกไฟล์จะเก็บเหตุการณ์ที่มีระดับของการล็อกเหมือนกันเอาไว้ โดยในเซิร์ฟเวอร์ LOG-CHAIN นี้ระดับของการล็อกเป็น info (LOG level info)

กฎที่สองจะทำการละทิ้งหมดทุกอย่างแพ็คเกจ เนื่องจากแพ็คเกจที่ถูกส่งต่อมายังเซิร์ฟเวอร์นี้เพื่อเก็บข้อมูลล็อกไฟล์ ซึ่งคือแพ็คเกจที่ถูกกำหนด TARGET จากไฟร์สเตชันไว้เป็น DROP นั่นเอง

สุดท้ายเมื่อได้กฎจากไฟร์สเตชันอยู่ในรูปแบบคำสั่งของไอฟีเทเบิลทั้งหมดแล้วก็จะทำการเขียนทั้งหมดลงบนไฟล์เชลล์สคริปต์ เพื่อทำการรันเพื่อใช้งานตามคำสั่งไอฟีเทเบิลดังกล่าว รวมทั้งเก็บไฟล์นี้ไว้ใช้ครั้งต่อไปหากโปรแกรมถูกปิดหรือเครื่องไฟร์สกรีนเริ่มทำงานใหม่เพื่อที่จะสามารถคงกฎเดิมไว้ได้

5.4.1.3 ส่วนที่ทำหน้าที่ส่งล็อกไปยังไฟร์สเตชัน

โปรแกรมในส่วนนี้จะทำการอ่านล็อกไฟล์จากเครื่องเกตเวย์และส่งไปเก็บยังแอ็กทิฟไดเรกทอรีบนไฟร์สเตชัน สามารถแสดงลักษณะการทำงานเป็นแผนภาพดังนี้



รูปที่ 5.4 แสดงการส่งล็อกของไฟร์สกรีน

เมื่อเริ่มการทำงานขึ้นมาโปรแกรมจะคอยตรวจสอบล็อกไฟล์เป็นระยะ และทำการส่งล็อกที่เกี่ยวข้องในส่วนของไอฟีเทเบิลกลับไปยังไฟร์สเตชัน

5.4.2 การทำงานร่วมกับไฟร์แตรป

หน้าที่ในส่วนของการทำงานร่วมกับไฟร์แตรปนั้น ไฟร์สกรีนจะรับแพ็คเก็ตที่ไม่ตรงตามกฎใด ๆ ที่ได้รับจากไฟร์สแตชัน แล้วนำมาจำแนกว่ามีพฤติกรรมเข้าข่ายเป็นผู้บุกรุกหรือไม่ โดยอาศัยระบบตรวจจับผู้บุกรุก ที่ชื่อสนอร์ตอินไลน์ (Snort_inline) ซึ่งเป็นเครื่องมือประเภท IDS ที่ทำงานอยู่ในระบบไฟร์สกรีนเพื่อให้มีความสามารถดังที่ได้กล่าวมา ในการทำงานของไฟร์สกรีนนั้นจะทำงานในรูปแบบของ Bridge Mode ทำให้ผู้ใช้งานที่ต้องการใช้งานเครื่องเซิร์ฟเวอร์จริง ไม่ทราบว่ามี การส่งข้อมูลผ่านตัวไฟร์สกรีน

และเมื่อระบบตรวจจับผู้บุกรุกในไฟร์สกรีนทำการตรวจจับแพ็คเก็ตที่เข้าข่ายเป็นผู้บุกรุกได้แล้ว โปรแกรม S2I ที่อยู่ในไฟร์สกรีนจะทำการเปลี่ยนเส้นทางให้ผู้บุกรุกเข้าไปใช้งานเครื่องกับดักในระบบไฟร์แตรปแทน ซึ่งเป็นเครื่องจำลองที่เตรียมไว้และมีสภาพแวดล้อมและค่าต่างๆ เหมือนเครื่องให้บริการจริง พร้อมทั้งส่งข้อมูลเกี่ยวกับการบุกรุกที่ตรวจจับได้กลับไปยังเครื่องแม่ข่ายฐานข้อมูลกลางของระบบ (LogServer) นอกจากนี้โปรแกรม S2I ยังทำหน้าที่ในการตรวจสอบความอ่อนแอของเครื่องกับดักในระบบไฟร์แตรป เช่น เครื่องกับดักถูกผู้บุกรุกทำการเปลี่ยนพาสเวิร์ด หรือเพิ่มผู้ใช้ใหม่หรือไม่ ซึ่งถ้าเครื่องกับดักถูกบุกรุกด้วยวิธีการดังกล่าว โปรแกรม S2I จะทำการหยุดการทำงานของเครื่องกับดักตัวนั้น พร้อมทั้งสั่งให้เครื่องกับดักตัวใหม่ขึ้นมาทำงานแทน

ในส่วนการทำงานของไฟร์สกรีนที่เป็นส่วนจำแนกผู้บุกรุกและทำงานร่วมกับระบบไฟร์แตรปตามที่ได้กล่าวมานี้ จะมีโปรแกรมควบคุมการทำงานจากศูนย์กลาง คือ โปรแกรม Tartarus Management ซึ่งเป็นโปรแกรมที่ควบคุมการทำงานของสนอร์ตอินไลน์, ควบคุมการทำงานของเครื่องกับดักในไฟร์แตรป รวมถึงการอัปเดต, สร้าง และเลือกใช้กฎต่างๆ ของสนอร์ตอินไลน์ โดยในการควบคุมระบบผ่านโปรแกรม Tartarus Management นี้ผู้ดูแลระบบจะกระทำผ่านช่องทางที่ปลอดภัยโดยใช้ IPsec (IPsecurity) ซึ่งจะมีการพิสูจน์ตัวตน (Authentication) และการเข้ารหัสข้อมูล (Encryption) ทำให้สามารถจัดการระบบผ่านเครือข่ายได้อย่างปลอดภัยมากยิ่งขึ้น

5.4.3 หลักการจำแนกเป็นผู้บุกรุก

ในการจำแนกผู้บุกรุกนั้นจะพิจารณาว่าผู้ใดเป็นผู้บุกรุกหรือเข้าข่ายเป็นผู้บุกรุกโดยใช้กฎของโปรแกรมตรวจจับผู้บุกรุก (Snort_inline) ซึ่งเป็นเครื่องมือประเภท IDS (Intrusion Detection System) โดยหากว่าการเชื่อมต่อใดมีลักษณะตรงตามกฎของโปรแกรมตรวจจับผู้บุกรุกจะถือว่าเจ้าของการเชื่อมต่อที่เข้าข่ายเป็นผู้บุกรุกจะต้องทำการเปลี่ยนทิศทางการเชื่อมต่อดังกล่าวไปยังเครื่องกับดักเพื่อดำเนินการบันทึกพฤติกรรมของผู้บุกรุกนั้นและเพื่อให้การจำแนกผู้บุกรุกของโปรแกรมตรวจจับผู้บุกรุกเป็นไปตามความต้องการของผู้พัฒนามากขึ้น ทางผู้พัฒนาจึงได้ทำการแก้ไขกฎการจำแนกบางส่วน เช่น

Mysql cazz exploit

กฎของการตรวจสอบการบุกรุกของเครื่องมือที่ใช้เข้าโจมตีโปรแกรมฐานข้อมูล ที่ชื่อว่ามายเอสคิวแอล (mysql) โดยชื่อโปรแกรมที่เรียกว่า cazz เพื่อเข้าโจมตีแบบยัดเครื่องระบบ เพื่อตั้งให้กฎเองมีความยืดหยุ่นที่ว่าหากมีการเชื่อมต่อเข้ามายังช่องทางหมายเลข 3306 ลักษณะการขอการสถาปนา ก็ให้ถือว่าเป็นการโจมตีจากผู้บุกรุก

```
alert tcp $EXTERNAL_NET any -> $SQL_SERVERS 3306
```

```
(msg:"MYSQL root login attempt";
```

```
flow:to_server,established; )
```

5.4.4 หลักการจำแนกกว่าเป็นผู้ใช้ปกติ

ในการจำแนกผู้ใช้ปกตินั้นจะพิจารณาโดยใช้กฎของโปรแกรมตรวจจับผู้บุกรุกเช่นเดียวกับการจำแนกผู้บุกรุก คือการเชื่อมต่อใดที่ไม่ตรงกับกฎให้ถือว่าเป็นเจ้าของการเชื่อมต่ออื่นๆ เป็นผู้ใช้ปกติสามารถให้ผ่านไปได้ใช้บริการของเครื่องแม่ข่ายจริงได้ตามปกติ และกรณีที่ไม่ต้องการให้มีการตรวจสอบเครื่องบางเครื่องที่เชื่อมต่อเข้ามา สามารถทำได้โดยการกำหนดกฎที่ ไฟร์วอลล์ให้การเชื่อมต่อสามารถผ่านเข้าไปได้โดยไม่ต้องมีการตรวจสอบเนื่องจากว่าผู้ที่ควบคุมการเชื่อมต่อจริงแล้วคือไฟร์วอลล์ ในการตรวจสอบของโปรแกรมตรวจจับผู้บุกรุกอาจมีความผิดพลาดได้จึงจำเป็นต้องตั้งกฎของระบบตรวจจับผู้บุกรุกให้มีความรัดกุมและบ่งบอกชัดเจนว่าผู้ใดเป็นผู้บุกรุกเพื่อให้กระบวนการตรวจสอบนั้นไม่เกิดผลการเข้าใจผิดว่า ผู้ใช้งานปกตินั้นเป็นผู้บุกรุก

5.4.5 หลักการจำแนกกว่าเป็นโค้ดหนอน

ในการระบุได้ว่าเป็นโค้ดหนอน จะใช้การพิจารณาแบบ Rules-base เช่นเดียวกับการตรวจจับผู้บุกรุก โดยจะใช้ฐานข้อมูลที่เก็บ Signature ของโค้ดหนอนชนิดต่างๆ เอาไว้ มาเปรียบเทียบกับรูปแบบที่พบ ซึ่งจำเป็นจะต้องมีการปรับปรุงฐานข้อมูล Signature นี้ให้ทันสมัยอยู่ตลอดเวลา ซึ่งทางผู้พัฒนาได้เพิ่มเข้าไปในส่วนของสนอร์ตอินไลน์เพื่อให้สามารถตรวจสอบโค้ดหนอนไปพร้อมกับตรวจจับผู้บุกรุกเลย